TASK 7 – Control Type Conversion and Precision:

7.1 Where conversions happen:

- When short or unsigned short operands are used in arithmetic, integer promotions usually convert them to int if int can represent every value of the original type.
- The usual arithmetic conversions decide the common type when different numeric types are combined.
- Integer division discards the fractional part when both operands are integers.
- An explicit cast can force conversion, but casting a value outside the destination type's range may lose information; converting an out-of-range signed integer to a smaller signed type is implementation-defined or may raise an implementation-defined signal.
- When converting fractional charges to paise, choose a documented rule, such as rounding to the nearest paise, and apply it once at the defined billing boundary.

7.2 Three expressions that can surprise:

Expression	Result	Explanation
5 / 2	2	Both operands are integers, so the fractional part is discarded.
(double)(5 / 2)	2.0	The division happens first as integer division; casting afterwards cannot recover the .5.
(double)5 / 2	2.5	One operand is converted before division, so floating-point division is used.
0.1 + 0.2 == 0.3	Often false in binary floating point	The decimal fractions are not represented exactly in binary; compare with a tolerance when floating point is appropriate.

7.3 Example of a narrow cast:

```
long long total_paise = 76000000000LL;
int narrowed = (int)total_paise; /* value is outside 32-bit int range */
printf("%d\n", narrowed);        /* do not rely on a particular result */
```

The example demonstrates why district totals must not be narrowed to int. Use a wide type for the total and match format specifiers to the exact type.

7.4 Float totals versus integer paise:

If all calculations were performed in float rupees and rounded only at the end, each charge would be subject to binary representation error and the running total would have limited precision. A float typically has about 6–7 decimal digits of precision. At large aggregate values, its spacing between representable numbers becomes much larger than one paisa, so a total can lose small increments.

Integer paise preserves exact cents as long as arithmetic does not overflow. For both approaches, adding n bills takes O(n) time overall and O(1) extra space for a running total. A float value commonly occupies 4 bytes; a long long commonly occupies 8 bytes. The integer approach is recommended for billing.